Community Scam Map

Hackathon brief: what we submit, how it works, and what judges may ask

1. What to submit (hosting is on localhost)

Item	Our answer	Status
Code in a Git repository	Branch **shubhang**; **scripts/check.sh** must pass.	Commit and push the open changes
Server public web address	Laptop on the judges' Wi-Fi: **http://<laptop-ip>:8000**. Printed by **scripts/run_local.sh**. Needs the firewall rule from infra/README.	Test from a second device. Plain http, not https: confirm with organisers
App: APK or mobile website	Mobile website (Streamlit) on port **8501**. No APK.	Done
README: run, trust rules, licence	README.md covers all three, plus known limits.	Done
Demo video, up to 3 min	Record: check a number, report, map and alerts, licence expiry and renewal.	**Not made yet**
Two licence files	Long: **.local/demo/licence-long.json** (valid to Nov 2027). 5-minute: sign it just before the demo.	5-minute file made on the day

Sign the 5-minute licence (its expiry time is absolute, so a file made earlier is already dead):

```
uv run --project api python scripts/license_sign.py --licensee "Demo Bank" --expires-in 5m --out
.local/demo/licence-5min.json
```

2. System design

Layer	What it is	Key point
Mobile website	Streamlit, port 8501. Screens: Check, Report, Map, Dispute, Licence, How it works. Live right-hand panel of alerts and events, refreshed every 3 s. English and Hindi.	Talks to the API only; holds no data
API	FastAPI, port 8000. Exactly the PRD routes: /health, /reports, /lookup, /map, /alerts, /admin/reset, /challenges, /license. Two UI-only routes: /app/areas, /app/activity.	No login, no proof-of-work; only 429 rate limits
Trust engine	Collapses repeats, merges copied notes, caps bursts, scores independent clusters, gives a verdict.	All thresholds in trust_config.py
Licence gate	Ed25519 signature checked on every request. Hot-swap with POST /license, no restart. Expiry returns 403 LICENSE_EXPIRED; data is never deleted.	Private key never on the server
Storage	SQLite (WAL). Reporter, network prefix and identifiers stored only as keyed HMAC; notes only as MinHash fingerprints.	No request log lines at all

Flow: phone → website → API (licence check → validate → hash identifiers → trust rules → store) → lookup, map and alerts read only **counted** reports.

3. Points division and where it is earned

PRD row	Pts	How we earn it	Proof
1 Stops fake reports	20	Reporter and network dedupe, copied-note merge, burst cap, 429 above 3 reports/hour	15 simulator families; held-out 750/750
2 Lookup is right	15	Verdict from independent clusters; masked until 3 reporters; verified list	Datasets; p95 latency tests
3 Map and alerts	10	Spike and new-type alerts on counted reports only; 30-day window	Map and alert families
4 Privacy	15	HMAC only, fingerprints only, no logs, careful wording	5 scanners: 0 findings; privacy check
5 Server and licence	15	Live 4-step demo; reachable address	Licence tests; **no https**, so partly proven
6 Usability	15	Icon-first mobile screens, under 20 s to report	**Judged by people; stopwatch test pending**
7 Demo and pitch	10	Clear 3-minute story	**Judged by people; video pending**

PRD row	Pts	How we earn it	Proof
Bonus	7	Verified numbers 3, disputes 2, fakes never alert 2	Contract checks

Total 100 + 7 bonus = 107. Our tools measure rows 1 to 5 and the bonus: 82.0 of 82. Rows 6 and 7 (25 points) depend on people, so the demo matters most.

4. Questions a recruiter or judge may ask

Why can't someone just spam reports to brand a number a scam?

One reporter counts once. Same network within 24 hours counts once. Copied notes merge. A burst of new reporters is capped as a single cluster. A known scam needs 3 independent clusters and a score of 3.0.

Why did you not use a login or proof-of-work?

The PRD forbids both on the report routes. We rate-limit with 429 and rely on the trust rules.

How do you protect privacy if you still need to spot repeats?

We keep keyed hashes, so the same value matches without being readable. Notes become fingerprints that detect similarity but cannot be read back. Nothing is logged.

What does 'verified business' mean?

Only that the value is on the organisers' Appendix C list. We do not check real-world identity.

What happens when the licence expires mid-demo?

Every route except /health and /license returns 403 LICENSE_EXPIRED. Uploading a new licence restores service at once with all data intact. A rolled-back clock also counts as expired.

Can the verdict be wrong?

Yes. A coordinated attack with many real-looking, varied reporters can still push a score, so we show counts, not accusations, and allow one dispute per number.

How do you know it works?

Seeded black-box simulator of 15 scenario families, tuned on tuning seeds and reported on held-out seeds only; five PRD-format datasets loaded with plain curl; 432 API, 197 web and 265 judge tests.

What is your weakest point?

Public https (we are on http over Wi-Fi), rows 6 and 7 not yet tested with real people, and the verified list being a static file.

5. Can judges inject data? How feasible?

There are no log files to inject into: by design we write **no per-request logs**, so nothing a judge sends can be read back from a log. Injecting **reports** is fully supported and is the intended test.

• Anyone on the Wi-Fi can call the PRD routes with plain **curl**: POST /reports, GET /lookup, /map, /alerts.

• **POST /admin/reset** wipes the data, as the PRD requires; **POST /license** swaps the licence. Neither needs a login, so keep the laptop on a trusted network.

• Verified: we erased the database and loaded all five datasets with curl only: 116 of 116 checks, 82.0 of 82.

• Limits: heavy load from one source gets 429, and a very large load is bounded by one laptop (SQLite).

6. Notable rule: reports from outside the city count for less

Idea: a report from outside a number's home city carries reduced weight, unless people from that city have also reported the same business or person.

• **Rule:** each report is tagged with its area's city. Out-of-city reports weigh about 0.5 until at least one in-city independent cluster exists; then they count fully.

• **Why:** a distant attacker cannot brand a local shop alone, but a real scam that travels (online, calls) is still confirmed once locals agree.

• **Feasibility:** high. Reports already carry an area (PRD Appendix B). It needs a city per area, one extra weight in trust_config.py, and tests. About half a day.

• **Status: not built yet.** It is a design proposal, and it needs the PRD to allow it for online-only scams (phone numbers and links have no city).

7. Future scope

• Public https hosting and a proper domain; an Android APK.

• Per-city trust weighting (above), and reporter reputation that does not need an identity.

• A review queue for disputes and a signed, updatable verified-business list.

• Move from SQLite to Postgres and add caching for many cities.

• More languages; push notifications for alerts near a user.

• Share confirmed scams with banks and telecom partners.

8. Demo essentials

• Before: run **scripts/run_local.sh**, test the laptop address from a phone, open the firewall rule, plug in power, stop sleep.

• Sign the 5-minute licence about one minute before the licence step.

• Order (3 minutes): check a known-scam number → report a new one → map and live panel → dispute → licence: upload 5-minute file, wait, see 403, upload long file, data intact.

• Say plainly: nothing is logged, personal details are hashed, wording is 'reported by N people', never 'scammer'.

• Backup: keep a recorded video and the curl commands in **.local/curl/**. Rehearse three times.

• Be honest about limits: http not https, verified means 'on the list'.